

圆

圆的反演：给定反演中心点 O 和反演半径 R 。若平面上点 P 和 P' 满足：点 P' 在射线 OP 上且 $||\vec{OP}|| \cdot ||\vec{OP'}|| = R^2$ ，则称点 P 和点 P' 互为反演点

性质1：圆 O 外的点的反演点在圆 O 内，反之亦然；圆 O 上的点的反演点为其自身

性质2：不过点 O 的圆 A ，其反演图形也是不过点 O 的圆

性质3：过点 O 的圆 A ，其反演图形是不过点 O 的直线

性质4：两个图形相切且存在不为点 O 的切点，则它们的反演图形也相切

```
1  #ifndef usedouble
2  // 圆的板子还没有全部测完，从俊杰板子直接偷的，稍微重构了一下
3  struct Circle {
4      Point c; // 圆心
5      T r;     // 半径
6
7      bool operator==(const Circle &a) const {return c==a.c && abs(r-a.r)<=eps;}
8      T circ() const {return 2 * pi * r;} // 圆的周长
9      T area() const {return pi * r * r;} // 圆的面积
10
11     // 点与圆的位置关系 -1: 圆上; 0: 圆外; 1: 圆内
12     int is_in(const Point &p) const {
13         const T d = p.dis(c);
14         return abs(d - r) <= eps ? -1 : d < r - eps;
15     }
16     // 直线与圆的位置关系 0: 相离; 1: 相切; 2: 相交
17     int relation(const Line &l) const {
18         const T d = l.dis(c);
19         if(d > r + eps) return 0;
20         if(abs(d - r) <= eps) return 1;
21         return 2;
22     }
23     // 两个圆的位置关系 -1: 完全重合; 0: 相离; 1: 外切; 2: 相交; 3: 内切; 4: 内含
24     int relation(const Circle &a) const {
25         if(*this == a) return -1;
26         const T d = c.dis(a.c);
27         if(d > r + a.r + eps) return 0;
28         if(abs(d - r - a.r) <= eps) return 1;
29         if(abs(d - abs(r - a.r)) <= eps) return 3;
30         if(d < abs(r - a.r) - eps) return 4;
31         return 2;
32     }
33     // 直线与圆的交点
```

```

34     vector<Point> inter(const Line &l) const {
35         const T d = l.dis(c);
36         const Point p = l.proj(c);
37         const int t = relation(l);
38         if(t == 0) return vector<Point>(); // 相离
39         if(t == 1) return vector<Point>{p}; // 相切
40         const T k = sqrt(r * r - d * d); // 相交
41         return vector<Point>{p - (l.v / l.v.len()) * k, p + (l.v / l.v.len())
* k};
42     }
43     // 两个圆的交点
44     vector<Point> inter(const Circle &a) const {
45         const T d = c.dis(a.c);
46         const int t = relation(a);
47         if(t == -1 || t == 0 || t == 4) return vector<Point>();
48         Point e = a.c - c; e = e / e.len() * r;
49         if(t == 1 || t == 3) {
50             if(r * r + d * d - a.r * a.r >= -eps) return vector<Point>{c + e};
51             return vector<Point>{c - e};
52         }
53         const T costh = (r * r + d * d - a.r * a.r) / (2 * r * d);
54         const T sinth = sqrt(1 - costh * costh);
55         return vector<Point>{c + e.rot(costh, -sinth), c + e.rot(costh,
sinth)};
56     }
57     // 两个圆相交部分的面积
58     T inter_area(const Circle &a) const {
59         const T d = c.dis(a.c);
60         const int t = relation(a);
61         if(t == -1) return area();
62         if(t < 2) return 0;
63         if(t > 2) return min(area(), a.area());
64         const T costh1 = (r * r + d * d - a.r * a.r) / (2 * r * d);
65         const T costh2 = (a.r * a.r + d * d - r * r) / (2 * a.r * d);
66         const T sinth1 = sqrt(1 - costh1 * costh1), sinth2 = sqrt(1 - costh2 *
costh2);
67         const T th1 = acos(costh1), th2 = acos(costh2);
68         return r * r * (th1 - costh1 * sinth1) + a.r * a.r * (th2 - costh2 *
sinth2);
69     }
70     // 过圆外一点 a 作该圆的切线, 返回过 a 的射线
71     vector<Line> tangent(const Point &a) const {
72         const int t = is_in(a);
73         if(t == 1) return vector<Line>();
74         if(t == -1) {
75             const Point v = {-(a - c).y, (a - c).x};
76             return vector<Line>{{a, v}};

```

```

77     }
78     Point e = a - c; e = e / e.len() * r;
79     const T costh = r / c.dis(a), sinth = sqrt(1 - costh * costh);
80     const Point t1 = c + e.rot(costh, -sinth), t2 = c + e.rot(costh,
sinth);
81     return vector<Line>{{a, t1 - a}, {a, t2 - a}};
82 }
83 // 求两圆的公切线，内切1条，相交2条，外切3条，相离4条
84 vector<Line> tangent(const Circle &a) const {
85     const int t = relation(a);
86     vector<Line> lines;
87     if(t == -1 || t == 4) return lines;
88     if(t == 1 || t == 3) {
89         const Point p = inter(a)[0], v = {-(a.c - c).y, (a.c - c).x};
90         lines.push_back({p, v});
91     }
92     const T d = c.dis(a.c);
93     const Point e = (a.c - c) / (a.c - c).len();
94     if(t <= 2) {
95         const T costh = (r - a.r) / d, sinth = sqrt(1 - costh * costh);
96         const Point d1 = e.rot(costh, -sinth), d2 = e.rot(costh, sinth);
97         const Point u1 = c + d1 * r, u2 = c + d2 * r, v1 = a.c + d1 * a.r,
v2 = a.c + d2 * a.r;
98         lines.push_back({u1, v1 - u1}); lines.push_back({u2, v2 - u2});
99     }
100     if(t==0) {
101         const T costh = (r + a.r) / d, sinth = sqrt(1 - costh * costh);
102         const Point d1 = e.rot(costh, -sinth), d2 = e.rot(costh, sinth);
103         const Point u1 = c + d1 * r, u2 = c + d2 * r, v1 = a.c - d1 * a.r,
v2 = a.c - d2 * a.r;
104         lines.push_back({u1, v1 - u1}); lines.push_back({u2, v2 - u2});
105     }
106     return lines;
107 }
108 // 圆的反演：直线的反演结果可能是圆或直线
109 std::variant<Circle, Line> inverse(const Line &l) const {
110     if(l.toleft(c) == 0) return l; // 经过反演中心的直线，反演后依然是原直线
111     const Point v = l.toleft(c) == 1 ? Point{l.v.y, -l.v.x} : Point{-
l.v.y, l.v.x};
112     const T d = r * r / l.dis(c);
113     const Point p = c + v / v.len() * d;
114     return Circle{(c + p) / 2, d / 2}; // 不经过反演中心的直线，反演后是过反演中
心的圆
115 }
116 // 圆的反演：圆的反演结果可能是圆或直线
117 std::variant<Circle, Line> inverse(const Circle &a) const {
118     const Point v = a.c - c;

```

```

119     if(a.is_in(c) == -1) { // 经过反演中心的圆，反演后是不经过反演中心的直线
120         const T d = r * r / (a.r + a.r);
121         const Point p = c + v / v.len() * d;
122         return Line{p, {-v.y, v.x}};
123     }
124     if(c==a.c) return Circle{c,r * r / a.r}; // 同心圆反演
125     // 不经过反演中心的圆，反演后也是不经过反演中心的圆
126     const T d1 = r * r / (c.dis(a.c) - a.r), d2 = r * r / (c.dis(a.c) +
a.r);
127     const Point p = c + v / v.len() * d1, q = c + v / v.len() * d2;
128     return Circle{(p + q) / 2, p.dis(q) / 2};
129 }
130 };
131
132 // 圆与多边形面积交
133 T area_inter(const Circle &circ, const Polygon &poly) {
134     const auto cal = [](const Circle &circ, const Point &a, const Point &b) {
135         if((a - circ.c).toleft(b - circ.c) == 0) return 0.01;
136         const auto ina = circ.is_in(a), inb = circ.is_in(b);
137         const Line ab = {a, b - a};
138         // 情况1: 两个端点都在圆内，直接算三角形面积
139         if(ina && inb) return ((a - circ.c) ^ (b - circ.c)) / 2;
140         // 情况2: 一个在内一个在外，由三角形加扇形组成
141         if(ina && !inb) {
142             const auto t = circ.inter(ab);
143             const Point p = t.size() == 1 ? t[0] : t[1];
144             const T ans = ((a - circ.c) ^ (p - circ.c)) / 2;
145             const T th = (p - circ.c).ang(b - circ.c);
146             const T d = circ.r * circ.r * th / 2;
147             if((a - circ.c).toleft(b - circ.c) == 1) return ans + d;
148             return ans - d;
149         }
150         if(!ina && inb) {
151             const Point p = circ.inter(ab)[0];
152             const T ans = ((p - circ.c) ^ (b - circ.c)) / 2;
153             const T th = (a - circ.c).ang(p - circ.c);
154             const T d = circ.r * circ.r * th / 2;
155             if((a - circ.c).toleft(b - circ.c) == 1) return ans + d;
156             return ans - d;
157         }
158         // 情况3: 两个端点都在圆外，如果线段和圆相交，为两扇形加一中间三角形
159         const auto p = circ.inter(ab);
160         if(p.size() == 2 && Segment{a, b}.dis(circ.c) <= circ.r + eps) {
161             const T ans = ((p[0] - circ.c) ^ (p[1] - circ.c)) / 2;
162             const T th1 = (a - circ.c).ang(p[0] - circ.c), th2 = (b -
circ.c).ang(p[1] - circ.c);

```

```

163         const T d1 = circ.r * circ.r * th1 / 2, d2 = circ.r * circ.r * th2
// 2;
164         if((a - circ.c).topleft(b - circ.c) == 1) return ans + d1 + d2;
165         return ans - d1 - d2;
166     }
167     // 情况4: 两 endpoints 在外且线段不穿过圆, 纯扇形
168     const T th = (a - circ.c).ang(b - circ.c);
169     if((a - circ.c).topleft(b - circ.c) == 1) return circ.r * circ.r * th /
2;
170     return -circ.r * circ.r * th / 2;
171 };
172 T ans = 0;
173 for(size_t i = 0; i < poly.p.size(); i++) {
174     const Point a = poly.p[i], b = poly.p[poly.nxt(i)];
175     ans += cal(circ, a, b);
176 }
177 return ans;
178 }
179
180 // 圆面积并, O(n^2 log n)
181 // 返回值: ans[i] 表示被至少覆盖了 i+1 次的区域的面积
182 vector<T> area_union(const vector<Circle> &circs) {
183     const size_t siz = circs.size();
184     using arc_t = tuple<Point, T, T, T>; // (圆心, 半径, 起始极角, 终止极角)
185     vector<vector<arc_t>> arcs(siz);
186     const auto eq = [] (const arc_t &u, const arc_t &v) {
187         const auto [u1, u2, u3, u4] = u;
188         const auto [v1, v2, v3, v4] = v;
189         return u1 == v1 && abs(u2 - v2) <= eps && abs(u3 - v3) <= eps &&
abs(u4 - v4) <= eps;
190     };
191     auto cut_circ = [&] (const Circle &ci, const size_t i) {
192         vector<pair<T, int>> evt;
193         evt.push_back({-pi, 0}); evt.push_back({pi, 0});
194         int init = 0;
195         for(size_t j = 0; j < circs.size(); j++) {
196             if(i == j) continue;
197             const Circle &cj = circs[j];
198             if(ci.r < cj.r - eps && ci.relation(cj) >= 3) init++;
199             const auto inters = ci.inter(cj);
200             if(inters.size() == 1) evt.push_back({atan2l((inters[0] - ci.c).y,
(inters[0] - ci.c).x), 0});
201             if(inters.size() == 2) {
202                 const Point dl = inters[0] - ci.c, dr = inters[1] - ci.c;
203                 T argl = atan2l(dl.y, dl.x), argr = atan2l(dr.y, dr.x);
204                 if(abs(argl + pi) <= eps) argl = pi;
205                 if(abs(argr + pi) <= eps) argr = pi;

```

```

206         if(argl > argr + eps) {
207             evt.push_back({argl, 1}); evt.push_back({pi, -1});
208             evt.push_back({-pi, 1}); evt.push_back({argr, -1});
209         } else {
210             evt.push_back({argl, 1});
211             evt.push_back({argr, -1});
212         }
213     }
214 }
215 sort(evt.begin(), evt.end());
216 int sum=init;
217 for(size_t i = 0; i < evt.size(); i++) {
218     sum += evt[i].second;
219     if(abs(evt[i].first - evt[i+1].first) > eps)
220         arcs[sum].push_back({ci.c, ci.r, evt[i].first, evt[i +
221 1].first});
222     if(abs(evt[i + 1].first - pi) <= eps) break;
223 }
224 };
225 const auto oint = [](const arc_t &arc) {
226     const auto [cc, cr, l, r] = arc;
227     if(abs(r - l - pi - pi) <= eps) return 2.01 * pi * cr * cr;
228     return cr * cr * (r - l) + cc.x * cr * (sin(r) - sin(l)) - cc.y * cr *
229 (cos(r) - cos(l));
230 };
231 for(size_t i = 0; i < circs.size(); i++) {
232     const auto &ci = circs[i];
233     cut_circ(ci, i);
234 }
235 vector<T> ans(siz);
236 for(size_t i = 0; i < siz; i++) {
237     T sum = 0;
238     sort(arcs[i].begin(), arcs[i].end());
239     int cnt = 0;
240     for(size_t j = 0; j < arcs[i].size(); j++) {
241         if(j > 0 && eq(arcs[i][j], arcs[i][j - 1]))
242             arcs[i + (++cnt)].push_back(arcs[i][j]);
243         else cnt = 0, sum += oint(arcs[i][j]);
244     }
245     ans[i] = sum / 2;
246 }
247 return ans;
248 }
249 #endif

```

